WEEK 4 · LESSON 23 OF 24 · HANDOUT

Linear Regression from Scratch

Study time: **about 55 minutes** (reading, video and activity)

▶ **Watch:** https://www.youtube.com/watch?v=7ArmBVF2dCs — open the lesson online to watch inside the course.

🎓 **Go deeper:** Harvard CS50 AI, Lecture 4: Learning (supervised, reinforcement, unsupervised) — https://www.youtube.com/watch?v=E4M_IQG0d9g

🎓 **Go deeper:** Stanford Machine Learning, Lecture 1 — Andrew Ng — https://www.youtube.com/watch?v=UzxYlbK2c7E

Your goal in this lesson: Build a linear regression model with your own gradient descent, measure its error, compare it with NumPy's answer, and evaluate it on test data.

The model

We predict a moto fare from the trip distance with a straight line:

> *predicted fare = w × distance + b*

- **w** (weight, the slope): extra RWF per kilometre.
- **b** (bias, the intercept): the starting fare.

Training means finding the w and b that make predictions as close as possible to the real fares.

Run the setup cell:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

rng = np.random.default_rng(42)
n = 300
distance = rng.uniform(1, 15, n).round(1)
rain = rng.random(n) < 0.25
hour = rng.integers(6, 23, n)
fare = (400 + 230 * distance + np.where(rain, 300, 0) + rng.normal(0, 150, n)).round(-1)
minutes = (distance * 2.8 + rng.normal(0, 3, n) + np.where(rain, 4, 0)).clip(3).round()
trips = pd.DataFrame({"distance_km": distance, "minutes": minutes, "fare_rwf": fare,
                      "hour": hour, "raining": rain})
```

Split into training and test data

```
idx = rng.permutation(n)
train, test = trips.iloc[idx[:240]], trips.iloc[idx[240:]]
x_train, y_train = train["distance_km"].to_numpy(), train["fare_rwf"].to_numpy()
x_test, y_test = test["distance_km"].to_numpy(), test["fare_rwf"].to_numpy()
print(len(train), len(test))
```

We learn from 240 trips and keep 60 aside to test honestly (AI Foundations, Lesson 13).

The loss: mean squared error

> *MSE = average of (prediction − real value)²*

```
def predict(x, w, b):
    return w * x + b

def mse(y_true, y_pred):
    return np.mean((y_pred - y_true) ** 2)

print(round(mse(y_train, predict(x_train, 0, 0))))   # a terrible starting guess
```

The gradients

Using the derivative rules from Lesson 21:

> *∂MSE/∂w = 2 × average((prediction − y) × x) ∂MSE/∂b = 2 × average(prediction − y)*

Gradient descent

```
w, b = 0.0, 0.0
lr = 0.01
history = []
for epoch in range(5000):
    error = predict(x_train, w, b) - y_train
    dw = 2 * np.mean(error * x_train)
    db = 2 * np.mean(error)
    w -= lr * dw
    b -= lr * db
    history.append(mse(y_train, predict(x_train, w, b)))

print(f"w = {w:.1f} RWF per km, b = {b:.1f} RWF")
```

w should come out at about **225 RWF per km** and b at about **520 RWF** — close to the rule used to simulate the data (230 RWF per km and a 400 RWF start). b is a little higher because about a quarter of the trips include a 300 RWF rain charge that this model cannot see. The model discovered the pricing rule from the examples alone.

```
plt.plot(history)
plt.yscale("log")
plt.title("Training loss (MSE)")
plt.xlabel("epoch")
plt.show()
```

Check against NumPy

NumPy can solve a straight-line fit directly:

```
w_np, b_np = np.polyfit(x_train, y_train, 1)
print(round(w_np, 1), round(b_np, 1))
```

Your gradient descent should agree to within about 1 RWF. For a straight line there is an exact formula; gradient descent matters because it also works for models with no exact formula — like neural networks.

Evaluate on the test set

```
pred_test = predict(x_test, w, b)
mae = np.mean(np.abs(pred_test - y_test))
r2 = 1 - np.sum((y_test - pred_test) ** 2) / np.sum((y_test - y_test.mean()) ** 2)
print(f"Test MAE: {mae:.0f} RWF   Test R²: {r2:.3f}")

plt.scatter(x_test, y_test, alpha=0.6, label="real fares (test)")
xs = np.linspace(1, 15, 50)
plt.plot(xs, predict(xs, w, b), color="red", label="model")
plt.xlabel("Distance (km)")
plt.ylabel("Fare (RWF)")
plt.legend()
plt.show()
```

- **MAE** — on average, the prediction is off by this many RWF.
- **R²** — the share of variation in fares the model explains (1 = perfect).

What the model cannot see

Part of the remaining error comes from **rain**: rainy trips cost about 300 RWF more, and our model only knows distance. Adding more features (multiple linear regression) is your final project.

Key terms

- **Linear regression** — predicting a number with a straight-line model w × x + b.
- **Weight / bias** — slope / intercept of the line.
- **MSE** — mean squared error, the loss we minimise.
- **Epoch** — one pass of gradient descent over the training data.
- **MAE** — mean absolute error, easy to explain in real units.
- **R²** — share of variation explained by the model.

Remember this: You built a real machine-learning model from scratch: a prediction function, a loss, gradients and gradient descent. It learned the fare rule from data and was checked honestly on test data.

Name: _____ Date: _____

Activity

Task 1

Change the learning rate to 0.03, then 0.001. What happens to training?

Task 2

Predict the fare for a 7.5 km trip with your model.

Task 3

Train a second model that predicts `minutes` from `distance_km`. How many minutes per kilometre does it learn?

Task 4

Compute the test MAE separately for rainy and dry trips. Which is worse, and why?

Self-check

Think about your answers here, then take the quiz online — it is marked instantly and explains every answer.

1. In the model fare = w × distance + b, what does w mean?
   - A. The starting fare
   - B. The number of trips
   - C. The error
   - D. Extra RWF per kilometre (the slope)
2. What loss did we minimise?
   - A. The correlation
   - B. Mean squared error (MSE)
   - C. The number of rows
   - D. The median
3. Why keep a test set aside?
   - A. To make MSE bigger
   - B. To check the model honestly on data it has never seen
   - C. Because NumPy needs it
   - D. To train faster
4. The test MAE is 160 RWF. What does that mean?
   - A. The fare is 160 RWF
   - B. 160 trips were wrong
   - C. The model is 160% accurate
   - D. Predictions are off by about 160 RWF on average
5. Why use gradient descent if np.polyfit gives the answer directly?
   - A. polyfit is wrong
   - B. Gradient descent also works for models with no exact formula, like neural networks
   - C. It is always faster
   - D. It needs no data
6. Part of the remaining error comes from rain. How could the model improve?
   - A. Add raining as a second feature (multiple regression)
   - B. Use fewer trips
   - C. Use a bigger learning rate
   - D. Delete rainy trips

© WAMUGII TECH SOLUTIONS · wamugii.kento.rw · Original course material — for enrolled students.